UML Diagram 4: State Machines

Purpose: State transitions for engine mode, capacity increase requests, and emergency transfers.

Status: Design-first (60% complete) — high-level states identified; exact transition guards and events TBD (G-15/B-3 blocker).

4.1 Engine Mode State Machine

Purpose: Control when emergency transfers and autonomous operations are permitted.

Status: INCOMPLETE — G-15/B-3 production blocker. Transition guards, concurrency controls, and operator APIs TBD.

```
stateDiagram-v2
    [*] --> STEADY_STATE : System initialization

    STEADY_STATE --> DR_EVENT_ACTIVE : declareDREvent(incidentId, approver)
    STEADY_STATE --> MAINTENANCE_MODE : enterMaintenanceMode(reason, approver)

    DR_EVENT_ACTIVE --> FAILBACK_PENDING : resolveDREvent(incidentId, approver)
    DR_EVENT_ACTIVE --> STEADY_STATE : cancelDREvent(incidentId, approver) [emergency
drill only]

    FAILBACK_PENDING --> STEADY_STATE : confirmFailbackComplete(readinessGate, ap-
prover)
    FAILBACK_PENDING --> DR_EVENT_ACTIVE : failbackAborted(reason, approver)

    MAINTENANCE_MODE --> STEADY_STATE : exitMaintenanceMode(approver)

    note right of STEADY_STATE
        ALLOWED:
        - Placement
        - Auto-increase (Phase A: approval-gated)
        - Sharing management
        - Quota validation
        FORBIDDEN:
        - Emergency transfers (all tiers)
    end note

    note right of DR_EVENT_ACTIVE
        ALLOWED:
        - All STEADY_STATE operations
        - Emergency transfers (Tier 1/2/3)
        - Forced consumer removal
        FORBIDDEN:
        - Routine capacity reductions
        GUARDS:
        - Active IncidentRecord required
        - Tier 2/3 require dual approval
    end note

    note right of FAILBACK_PENDING
        ALLOWED:
        - Capacity restoration
        - VM re-association
        - Sharing restoration
        FORBIDDEN:
        - New placements
        - Emergency transfers
        READINESS GATE:
        - Prod region health confirmed
        - DR CRG capacity released
        - No active transfers
    end note

    note right of MAINTENANCE_MODE
        ALLOWED:
        - Read-only operations
        - Manual operator mutations
        FORBIDDEN:
        - Auto-increase
        - Reconciliation drift revert
        PURPOSE:
        - Scheduled maintenance
```

```
        - Break-glass manual intervention
    end note
```

Known Gaps — Engine Mode (G-15/B-3):

Transition Guards (TBD):
- `declareDREvent`: Requires `incidentId` + dual approval? Or single approver for SEV1?
- `resolveDREvent`: All active transfers must be `COMPLETED` or `ROLLED_BACK`? Or can resolve with transfers in-flight?
- `confirmFailbackComplete`: Exact readiness gate criteria TBD (health check? manual validation?)
- `enterMaintenanceMode`: Who can trigger? Time-bounded role? Require explicit exit?

Concurrency Controls (TBD):
- Can multiple incidents trigger concurrent DR events? Or one active incident at a time?
- If `declareDREvent` called while already in `DR_EVENT_ACTIVE`, is it idempotent or error?
- Lock acquisition: Does mode transition acquire a distributed lock to prevent concurrent state changes?

Recovery Rules (TBD):
- If system crashes mid-transition, how is mode recovered? Cosmos DB read? Event log replay?
- If mode is `DR_EVENT_ACTIVE` but incident is externally resolved (ITSM ticket closed), does engine auto-transition? Or require operator confirmation?

Operator APIs (TBD):
- FastAPI endpoint signatures for `declareDREvent`, `resolveDREvent`, `confirmFailbackComplete` TBD
- Approval workflow: Inline API parameter? Or external approval system integration?
- Audit trail: Every mode transition logged to `AuditEvent` with approver identity?

4.2 Capacity Increase Request Lifecycle

Purpose: Track steady-state and emergency capacity increase workflows.

```
stateDiagram-v2
    [*] --> PENDING_APPROVAL : Auto-increase threshold triggered OR Manual request

    PENDING_APPROVAL --> APPROVED : approveIncrease(approverId)
    PENDING_APPROVAL --> DENIED : denyIncrease(approverId, reason)
    PENDING_APPROVAL --> CANCELLED : cancelRequest(requesterId)

    APPROVED --> IN_PROGRESS : executeIncrease()

    IN_PROGRESS --> QUOTA_INCREASED : Quota action completed (if needed)
    IN_PROGRESS --> FAILED : Quota increase failed

    QUOTA_INCREASED --> CRG_UPDATED : ARM update CRG quantity
    QUOTA_INCREASED --> FAILED : CRG update failed

    CRG_UPDATED --> COMPLETED : ARM confirms state
    CRG_UPDATED --> FAILED : ARM state mismatch

    FAILED --> PENDING_APPROVAL : retry(requesterId) [manual retry only]

    COMPLETED --> [*]
    DENIED --> [*]
    CANCELLED --> [*]
    FAILED --> [*] : After max retries

    note right of PENDING_APPROVAL
        Phase A (current):
        - Operator approval required
        Phase B (future):
        - Auto-approve if quota headroom sufficient
    end note

    note right of IN_PROGRESS
        Multi-step saga:
        1. Pre-validate quota
        2. Request quota increase (if needed)
        3. Wait for quota confirmation
        4. Update CRG quantity via ARM
        5. Confirm ARM state
        Compensation if step N fails:
        - Release quota reservation
        - Restore CRG to prior quantity
    end note

    note right of FAILED
        Failure reasons:
        - Quota request denied
        - ARM throttled
        - ARM state divergence
        - Timeout waiting for quota
        Manual intervention required:
        - Review failure reason
        - Retry or cancel
    end note
```

Known Gaps — Increase Request:

Approval Logic (Phase A vs B):

- Phase A: Every request requires explicit operator approval. Approval timeout? Auto-deny after N hours?

- Phase B: Auto-approve only when `quota_headroom >= demandVCPUs`. Exact formula TBD.
- Can operator override auto-denial in Phase B?

Quota Increase Workflow:
- Is quota increase request automated (Azure Support API)? Or manual ticket creation?
- If manual, how does system know when quota is granted? Polling ARM? Or operator manual confirmation?
- Timeout for quota grant: 24h? 48h? SLA varies by region/SKU.

Retry Strategy:
- Max retries: 3? 5? Configurable?
- Backoff: Exponential? Fixed interval?
- Dead-letter: After max retries, send to manual queue?

Saga Compensation:
- If quota was granted but CRG update failed, is quota “returned” via another quota decrease request? Or left as headroom?
- If CRG quantity was increased but ARM state check failed, is CRG reverted? Or left as-is pending reconciliation?

4.3 Emergency Transfer Workflow (Tier 1 / Tier 2 / Tier 3)

Purpose: Track destructive capacity transfer lifecycle.

```
stateDiagram-v2
    [*] --> INITIATED : initiateTransfer(tier, sourceRegion, targetRegion)

    INITIATED --> PRECONDITIONS_FAILED : Validation failed
    INITIATED --> QUOTA_VALIDATED : Pre-checks passed

    QUOTA_VALIDATED --> AWAITING_APPROVAL : [Tier 2 or Tier 3]
    QUOTA_VALIDATED --> REDUCING_SOURCE : [Tier 1, auto-approved]

    AWAITING_APPROVAL --> APPROVED_TIER2 : Dual approval granted [Tier 2]
    AWAITING_APPROVAL --> APPROVED_TIER3 : Dual approval granted [Tier 3]
    AWAITING_APPROVAL --> DENIED : Approval denied
    AWAITING_APPROVAL --> CANCELLED : Operator cancels

    APPROVED_TIER2 --> REDUCING_SOURCE
    APPROVED_TIER3 --> DISASSOCIATING_VMS

    REDUCING_SOURCE --> EXPANDING_TARGET : Source CRG quantity reduced
    REDUCING_SOURCE --> COMPENSATING : Source reduction failed

    DISASSOCIATING_VMS --> REDUCING_SOURCE : VMs disassociated (Path A or B)
    DISASSOCIATING_VMS --> COMPENSATING : VM disassociation failed

    EXPANDING_TARGET --> COMPLETED : Target CRG quantity increased
    EXPANDING_TARGET --> COMPENSATING : Target expansion failed

    COMPENSATING --> ROLLED_BACK : Compensation completed
    COMPENSATING --> FAILED : Compensation failed

    COMPLETED --> [*]
    DENIED --> [*]
    CANCELLED --> [*]
    PRECONDITIONS_FAILED --> [*]
    ROLLED_BACK --> [*]
    FAILED --> [*] : Manual intervention required

    note right of INITIATED
        Precondition checks:
        - engine_mode == DR_EVENT_ACTIVE
        - Active IncidentRecord exists
        - Source/target regions valid
        - Source CRG has sufficient capacity
        Tier 1: Headroom available
        Tier 2: Quota-neutral possible
        Tier 3: VMs eligible for disassociation
    end note

    note right of DISASSOCIATING_VMS
        Tier 3 only:
        1. Select VMs (Dev → Test → Staging)
        2. Order by vCPU (smallest first)
        3. Path B (default): Reduce provider CR → Clear VM CRG ref
        4. Path A (fallback): Deallocate VM
        Every VM impact logged to VM_ImpactRecord
        Phase 1: VMSS blocked (explicit reject)
    end note

    note right of COMPENSATING
        Rollback sequence:
        - If VM disassociation done: re-associate VMs
        - If source reduced: restore source CRG quantity
        - If target expanded: reduce target CRG quantity
```

```
        - Release quota reservations
        All steps logged to OperationRecord compensation chain
    end note

    note right of FAILED
        Manual intervention required:
        - Compensation itself failed
        - Partial state (e.g., VMs disassociated but CRG update failed)
        - Operator must review and manually reconcile
        SOP TBD
    end note
```

Known Gaps — Transfer Workflow:

Tier Escalation:
- Does Tier 1 failure auto-escalate to Tier 2? Or require operator decision?
- If Tier 2 approved but fails (e.g., ARM throttled), can operator retry or must re-request approval?
- Tier 3 fallback: If Path B fails, does system auto-retry Path A? Or require operator approval?

Dual Approval Mechanism:
- Exact workflow TBD: Two approvers via API? ITSM ticket workflow? Dedicated UI?
- Approval timeout: Auto-deny after N hours? Or pending indefinitely?
- Approver eligibility: RBAC role? Pre-configured approver list?

Compensation Edge Cases:
- If VM re-association (compensation) fails, is VM left orphaned? Or manual SOP?
- If target CRG expansion succeeded but source reduction failed (inconsistent state), does compensation revert target? Or leave as-is?
- If compensation itself times out (ARM throttled), how long to retry before escalating to manual intervention?

Cross-Subscription VM Access (G-14 blocker):
- How does ACRME get write permissions to consumer-owned VMs for Tier 3 disassociation?
- User-assigned Managed Identity (UAMI) delegated per consumer subscription? Service Principal with certificates? User-provided credentials?
- Credential revocation: If consumer revokes access mid-transfer, does transfer fail gracefully?

VMSS Phase 1 Limitation:
- How is VMSS detected? Resource type check (`Microsoft.Compute/virtualMachineScaleSets`)? Or explicit VMSS list in config?
- If VMSS detected in VM list for Tier 3, does transfer fail entire operation? Or skip VMSS and proceed with standalone VMs?

Design Session Next Steps (to resolve gaps):

G-15/B-3: Engine Mode State Machine

1. **Define authoritative transition table** with guards, events, and approver requirements
2. **Concurrency control mechanism**: Distributed lock (Redis)? Optimistic versioning (Cosmos DB etag)?
3. **Recovery on crash**: Event sourcing? Or read-from-Cosmos on startup?
4. **Operator API contracts**: FastAPI endpoint signatures, request/response DTOs

5. **Audit trail**: Every transition logged with approver identity, timestamp, reason

IncreaseRequest Lifecycle

1. **Phase A → Phase B transition criteria**: Exact headroom threshold, approval bypass conditions
2. **Quota increase automation**: Azure Support API integration? Or manual SOP?
3. **Retry strategy**: Max retries, backoff algorithm, dead-letter handling
4. **Compensation edge cases**: Quota granted but CRG update failed — revert quota? Leave as headroom?

Transfer Workflow

1. **Tier escalation decision tree**: Auto-escalate vs. operator-gated
2. **Dual approval workflow**: ITSM integration? Inline API? Timeout policy?
3. **Compensation failure SOP**: Manual reconciliation steps, runbook
4. **G-14 resolution**: Select credential model (UAMI vs. Service Principal), test in consumer subscription, security approval
5. **VMSS rejection logic**: Detection mechanism, error message, Phase 2 roadmap for VMSS support

Cross-Cutting Concerns

1. **State persistence**: All state machines → Cosmos DB entities (EngineModeState, IncreaseRequest, EmergencyTransfer)
2. **Event sourcing**: Optional — emit state change events to Event Grid for downstream consumers?
3. **Idempotency**: Every state transition must be retryable (e.g., calling `declareDREvent` twice with same `incidentId` is safe)
4. **Time-to-live (TTL)**: `COMPLETED`, `ROLLED_BACK`, `FAILED` states — archive after N days? Or retain indefinitely for audit?